

T06

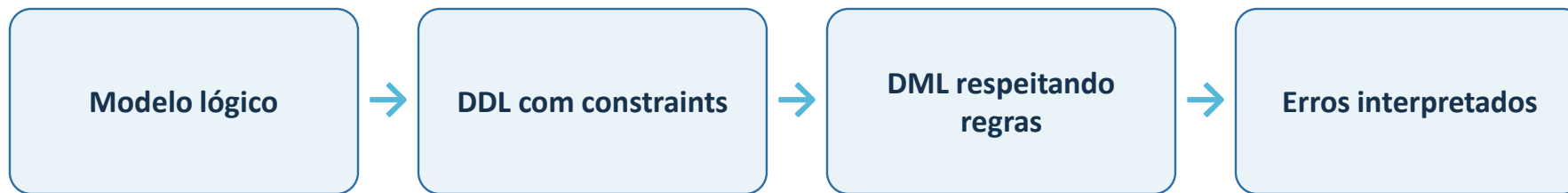
SQL no MySQL: DDL e DML

Da estrutura protegida às operações que criam e modificam dados

CREATE • ALTER • DROP • TRUNCATE • INSERT • UPDATE • DELETE

Prof. Calvetti

A T05 definiu as regras; a T06 executa mudanças controladas



Ao final da T06, você deverá ser capaz de...

Construir

Criar schemas e tabelas e alterar estruturas com DDL.

Manipular

Inserir, atualizar e excluir dados respeitando dependências.

Diagnosticar

Ler erros de integridade e prevenir mudanças de grande alcance.

Três aulas, um caso condutor e aplicação progressiva

1ª aula

CREATE DATABASE, USE, CREATE TABLE,
ALTER TABLE, DROP TABLE e TRUNCATE
TABLE.

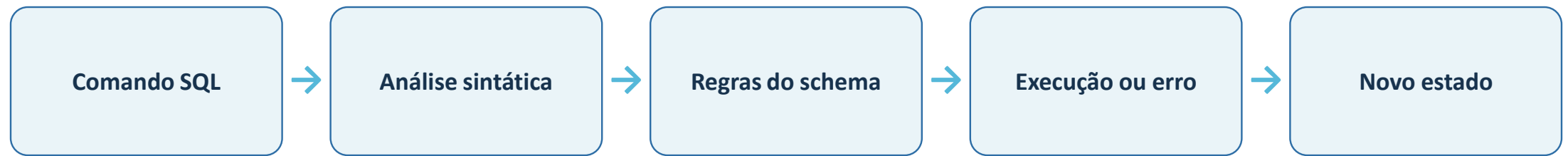
2ª aula

Constraints, índices associados às
chaves e AUTO_INCREMENT.

3ª aula

INSERT, inserção múltipla, UPDATE,
DELETE e riscos sem WHERE.

SQL declarativa descreve o estado desejado e o SGBD protege a transição



01

DDL: definir e evoluir estruturas

DDL — Data Definition Language:

DDL cria, modifica e remove objetos. Cada comando deve ser lido como uma mudança de estado no catálogo do banco.

DDL e DML atuam em objetos diferentes

DDL

Define schemas, tabelas, colunas, constraints e índices.

DML

Insere, atualiza e exclui linhas das tabelas.

Ponto comum

Ambas podem falhar quando violam sintaxe, dependências ou integridade.

Criar o schema estabelece o espaço de nomes do domínio

```
CREATE DATABASE IF NOT EXISTS loja_academica  
CHARACTER SET utf8mb4  
COLLATE utf8mb4_0900_ai_ci;
```

Leia o comando por cláusulas; execute e interprete o resultado.

Selecionar o schema evita operar no contexto errado

```
USE loja_academica;
```

```
SELECT DATABASE();
```

```
-- Confirme o contexto antes de executar DDL ou DML.
```

Leia o comando por cláusulas; execute e interprete o resultado.

A anatomia do comando combina colunas e regras

```
CREATE TABLE cliente (  
  id_cliente INT NOT NULL AUTO_INCREMENT,  
  nome VARCHAR(120) NOT NULL,  
  email VARCHAR(254) NOT NULL,  
  CONSTRAINT pk_cliente PRIMARY KEY (id_cliente),  
  CONSTRAINT uq_cliente_email UNIQUE (email)  
) ENGINE = InnoDB;
```

Leia o comando por cláusulas; execute e interprete o resultado.

Tabelas referenciadas devem existir antes das tabelas filhas



CLIENTE nasce com identidade e e-mail alternativo

```
CREATE TABLE cliente (  
  id_cliente INT NOT NULL AUTO_INCREMENT,  
  nome VARCHAR(120) NOT NULL,  
  email VARCHAR(254) NOT NULL,  
  ativo BOOLEAN NOT NULL DEFAULT TRUE,  
  CONSTRAINT pk_cliente PRIMARY KEY (id_cliente),  
  CONSTRAINT uq_cliente_email UNIQUE (email),  
  CONSTRAINT ck_cliente_ativo CHECK (ativo IN (0,1))  
) ENGINE = InnoDB;
```

Leia o comando por cláusulas; execute e interprete o resultado.

PRODUTO protege preço e estoque desde a criação

```
CREATE TABLE produto (  
  id_produto INT NOT NULL AUTO_INCREMENT,  
  nome VARCHAR(160) NOT NULL,  
  preco DECIMAL(10,2) NOT NULL,  
  estoque INT NOT NULL DEFAULT 0,  
  CONSTRAINT pk_produto PRIMARY KEY (id_produto),  
  CONSTRAINT ck_produto_preco CHECK (preco >= 0),  
  CONSTRAINT ck_produto_estoque CHECK (estoque >= 0)  
) ENGINE = InnoDB;
```

Leia o comando por cláusulas; execute e interprete o resultado.

PEDIDO referencia CLIENTE e controla seu estado

```
CREATE TABLE pedido (  
  id_pedido INT NOT NULL AUTO_INCREMENT,  
  criado_em DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  status VARCHAR(20) NOT NULL DEFAULT 'ABERTO',  
  id_cliente INT NOT NULL,  
  CONSTRAINT pk_pedido PRIMARY KEY (id_pedido),  
  CONSTRAINT ck_pedido_status CHECK (status IN ('ABERTO','PAGO','CANCELADO')),  
  CONSTRAINT fk_pedido_cliente FOREIGN KEY (id_cliente)  
    REFERENCES cliente(id_cliente) ON DELETE RESTRICT  
) ENGINE = InnoDB;
```

Leia o comando por cláusulas; execute e interprete o resultado.

ITEM_PEDIDO materializa o N:N entre pedidos e produtos

```
CREATE TABLE item_pedido (  
  id_pedido INT NOT NULL,  
  id_produto INT NOT NULL,  
  quantidade INT NOT NULL,  
  preco_praticado DECIMAL(10,2) NOT NULL,  
  CONSTRAINT pk_item PRIMARY KEY (id_pedido,id_produto),  
  CONSTRAINT ck_item_qtd CHECK (quantidade > 0),  
  CONSTRAINT fk_item_pedido FOREIGN KEY (id_pedido)  
    REFERENCES pedido(id_pedido) ON DELETE CASCADE,  
  CONSTRAINT fk_item_produto FOREIGN KEY (id_produto)  
    REFERENCES produto(id_produto) ON DELETE RESTRICT  
) ENGINE = InnoDB;
```

Leia o comando por cláusulas; execute e interprete o resultado.

ALTER TABLE evolui uma estrutura já existente

Adicionar

Nova coluna, constraint ou índice.

Modificar

Tipo, nulidade, padrão ou nome de coluna.

Remover

Coluna, chave estrangeira, índice ou outra regra nomeada.

ADD COLUMN introduz um novo atributo

```
ALTER TABLE produto  
  ADD COLUMN categoria VARCHAR(40)  
  AFTER nome;
```

-- Antes: examine linhas existentes e a regra de nulidade.

Leia o comando por cláusulas; execute e interprete o resultado.

MODIFY altera a definição; CHANGE também pode renomear

```
ALTER TABLE produto  
  MODIFY COLUMN categoria VARCHAR(60) NOT NULL;  
  
ALTER TABLE produto  
  CHANGE COLUMN preco preco_atual DECIMAL(12,2) NOT NULL;
```

Leia o comando por cláusulas; execute e interprete o resultado.

Constraints também podem ser adicionadas posteriormente

```
ALTER TABLE produto  
  ADD CONSTRAINT uq_produto_nome UNIQUE (nome),  
  ADD CONSTRAINT ck_produto_preco  
    CHECK (preco_atual >= 0);
```

Leia o comando por cláusulas; execute e interprete o resultado.

Remover uma regra exige seu nome e sua natureza

```
ALTER TABLE pedido  
DROP FOREIGN KEY fk_pedido_cliente;
```

```
ALTER TABLE produto  
DROP INDEX uq_produto_nome;
```

```
ALTER TABLE produto  
DROP COLUMN categoria;
```

Leia o comando por cláusulas; execute e interprete o resultado.

DROP TABLE remove a estrutura e seus dados

```
DROP TABLE IF EXISTS item_pedido;
```

- Dependências importam: filhos devem ser tratados antes dos pais.
- Em geral, DDL não pode ser desfeita com ROLLBACK.

Leia o comando por cláusulas; execute e interprete o resultado.

TRUNCATE esvazia a tabela preservando sua definição

```
TRUNCATE TABLE produto;
```

- Remove todas as linhas.
- É DDL, exige privilégio DROP e não aceita WHERE.
- FKs podem impedir a operação.

Leia o comando por cláusulas; execute e interprete o resultado.

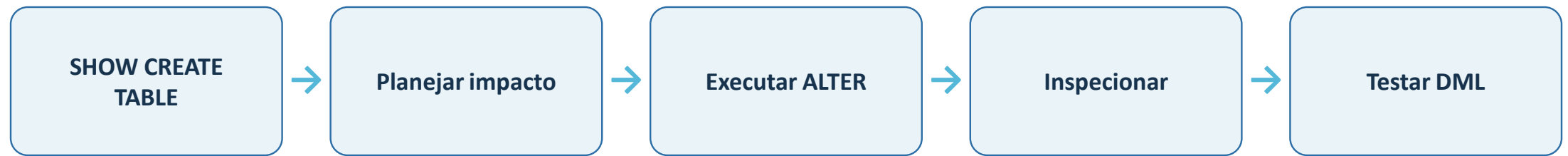
DELETE, TRUNCATE e DROP têm alcances distintos

Comando	Remove	Preserva
DELETE ... WHERE	Linhas selecionadas	Tabela e demais linhas
DELETE sem WHERE	Todas as linhas	Estrutura da tabela
TRUNCATE TABLE	Todas as linhas	Estrutura da tabela
DROP TABLE	Dados e definição	Nada da tabela

Os comandos DDL percorrem estados diferentes do objeto



Mudanças estruturais pedem diagnóstico antes e verificação depois



“Cannot add foreign key constraint” é sintoma, não diagnóstico final

- A tabela ou coluna referenciada não existe.
- Tipos das colunas não são compatíveis.
- A chave referenciada não possui índice adequado.
- Há dados existentes que violam a nova FK.
- O nome da constraint conflita ou o mecanismo não oferece suporte esperado.

02

Constraints, índices e AUTO_INCREMENT

Chaves criam garantias lógicas e estruturas de acesso. Compreender a relação entre elas evita índices redundantes e falsas garantias.

Constraints respondem a perguntas diferentes

Regra	Protege	Exemplo
PRIMARY KEY	Identidade única e não nula	id_pedido
FOREIGN KEY	Referência existente	id_cliente
UNIQUE	Não repetição	email
NOT NULL	Obrigatoriedade	nome
DEFAULT	Valor quando omitido	ABERTO
CHECK	Predicado do domínio	quantidade > 0

Chaves e índices se relacionam, mas não são sinônimos

PRIMARY KEY

índice único
clusterizado no InnoDB

UNIQUE

índice único
protege duplicidade

FOREIGN KEY

índice de apoio
nas colunas da FK

INDEX

acesso e ordenação
não cria regra de
domínio

Restrição e estrutura de acesso se relacionam, mas não são sinônimos.

No InnoDB, a chave primária organiza o índice clusterizado

Regra lógica

Impede identidade duplicada ou nula.

Estrutura física

Os dados são organizados pela chave primária.

Projeto

Declare uma PK curta, estável e coerente para cada tabela.

UNIQUE cria um índice que rejeita valores repetidos

Uso

E-mail, matrícula e combinações alternativas.

NULL

Múltiplos NULL podem ser aceitos;
combine com NOT NULL quando necessário.

Atenção

Índice comum acelera acesso, mas não impede duplicidade.

A verificação referencial depende de índices compatíveis

Filha

O MySQL requer índice nas colunas da FK; pode criá-lo automaticamente.

Pai

A chave referenciada precisa estar indexada de forma compatível.

Disciplina

Nomeie constraints e inspecione os índices efetivamente criados.

Um INDEX comum melhora acesso, mas não valida o domínio

```
CREATE INDEX ix_pedido_criado_em  
ON pedido (criado_em);
```

- Acelera certos acessos.
- Não garante unicidade.
- Não substitui PRIMARY KEY, UNIQUE ou FOREIGN KEY.

Leia o comando por cláusulas; execute e interprete o resultado.

SHOW INDEX revela as estruturas de acesso da tabela

```
SHOW INDEX FROM pedido;  
SHOW CREATE TABLE pedido;
```

- Compare Key_name, Non_unique e a ordem das colunas.
- Confirme o que foi criado explicitamente ou como apoio às chaves.

Leia o comando por cláusulas; execute e interprete o resultado.

AUTO_INCREMENT gera identidades; não numeração contábil

Geração

Ao omitir a coluna, o MySQL atribui um novo inteiro.

Lacunas

Erros, concorrência e exclusões podem deixar intervalos.

Semântica

O valor identifica; não representa posição, quantidade ou sequência sem falhas.

A aplicação recupera a identidade gerada na sessão

```
INSERT INTO cliente (nome,email)
VALUES ('Ana Lima','ana@exemplo.br');

SELECT LAST_INSERT_ID();

-- Use o id retornado para inserir registros dependentes.
```

Leia o comando por cláusulas; execute e interprete o resultado.

A ordem das colunas importa no índice composto

PK do item

PRIMARY KEY (id_pedido, id_produto).

Prefixo esquerdo

Consultas por id_pedido aproveitam o início do índice.

Outro acesso

Consultas apenas por id_produto podem exigir índice adicional.

Adicionar UNIQUE pode falhar se os dados já estiverem duplicados

```
SELECT email, COUNT(*)  
FROM cliente  
GROUP BY email  
HAVING COUNT(*) > 1;  
  
ALTER TABLE cliente  
ADD CONSTRAINT uq_cliente_email UNIQUE (email);
```

Leia o comando por cláusulas; execute e interprete o resultado.

Leia o erro à luz da regra que o comando tentou criar

Situação	Causa provável	Ação
Duplicate entry	PK ou UNIQUE repetida	Localizar e corrigir duplicatas
Cannot add FK	Definição ou dados incompatíveis	Comparar tipos, índices e órfãos
Check constraint	Linhas violam predicado	Corrigir dados ou regra
Multiple auto column	AUTO_INCREMENT inválido	Manter uma coluna indexada adequada

Índice redundante: diagnosticar antes de criar

Definição existente	Proposta	Decisão
PRIMARY KEY(id_pedido,id_produto)	INDEX(id_pedido)	Redundante pelo prefixo esquerdo
PRIMARY KEY(id_pedido,id_produto)	INDEX(id_produto)	Pode ser útil para acesso por produto
UNIQUE(email)	INDEX(email)	Em geral redundante
FOREIGN KEY(id_cliente)	INDEX(id_cliente)	Verificar o índice já criado

Inspecione as chaves de MATRÍCULA

- Use PK(id_matricula) e UNIQUE(id_aluno,id_turma).
- Declare FKs para ALUNO e TURMA.
- Liste os índices que você espera encontrar.
- Indique quais índices adicionais seriam redundantes.
- Justifique um índice para consultas de todas as matrículas de uma turma.

03

DML: inserir, atualizar e excluir linhas

DML — Data Manipulation Language

DML muda o conteúdo sem redefinir a tabela. O alvo de cada operação deve ser explícito e compatível com todas as constraints.

INSERT declara a tabela, as colunas e os valores da nova linha

```
INSERT INTO cliente (nome, email)  
VALUES ('Ana Lima', 'ana@exemplo.br');
```

-- Liste as colunas: reduz ambiguidades e resiste melhor a evoluções do schema.

Leia o comando por cláusulas; execute e interprete o resultado.

DEFAULT e NULL não significam a mesma coisa

Coluna omitida

Usa DEFAULT, geração automática ou regra do modo SQL.

NULL explícito

Só é aceito se a coluna admitir NULL.

DEFAULT explícito

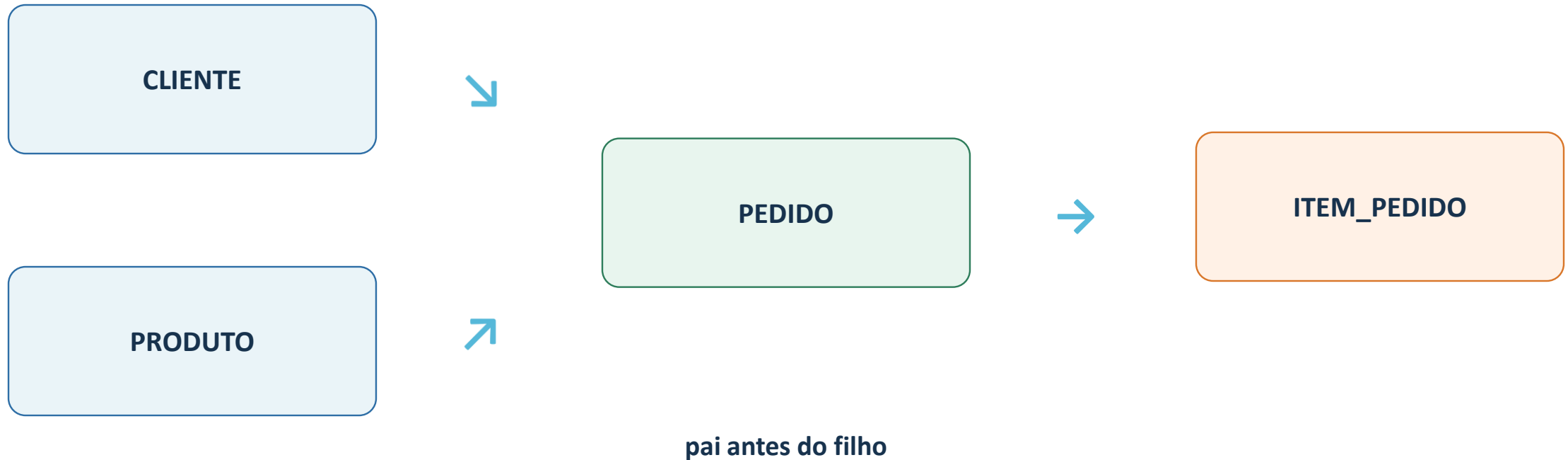
A palavra DEFAULT solicita o padrão definido para a coluna.

Uma instrução pode inserir várias linhas

```
INSERT INTO produto (nome, preco_atual, estoque) VALUES  
('Livro de SQL', 89.90, 12),  
('Caderno', 18.50, 40),  
('Caneta', 5.20, 80);
```

Leia o comando por cláusulas; execute e interprete o resultado.

Povoar a base segue a direção pai antes do filho



O id gerado conecta o novo cliente ao pedido

```
INSERT INTO cliente (nome,email)
VALUES ('Bruno Reis','bruno@exemplo.br');
SET @id_cliente = LAST_INSERT_ID();

INSERT INTO pedido (id_cliente)
VALUES (@id_cliente);
```

Leia o comando por cláusulas; execute e interprete o resultado.

Cada violação aponta para uma hipótese do modelo

Tentativa	Constraint	Resultado
E-mail repetido	UNIQUE	Rejeita a nova linha
Preço negativo	CHECK	Rejeita o valor
Pedido sem cliente	FOREIGN KEY	Rejeita a referência órfã
Nome omitido	NOT NULL	Rejeita a ausência
Status omitido	DEFAULT	Insere ABERTO

UPDATE define novos valores e seleciona as linhas-alvo

```
UPDATE produto  
SET preco_atual = preco_atual * 1.05  
WHERE categoria = 'LIVRO';  
  
-- SET descreve a mudança; WHERE delimita o alcance.
```

Leia o comando por cláusulas; execute e interprete o resultado.

WHERE é parte da regra de segurança, não mero detalhe sintático

COM WHERE

```
UPDATE produto  
SET preco = preco * 1.05  
WHERE categoria = 'LIVRO';
```

SEM WHERE

linha

linha

linha

linha

linha

linha

linha

linha

linha

linha

linha

linha

todas as linhas entram no alcance

Atualizar várias linhas pode ser correto quando o conjunto é consciente

```
SELECT id_produto, nome, estoque  
FROM produto  
WHERE estoque = 0;
```

```
UPDATE produto  
SET ativo = FALSE  
WHERE estoque = 0;
```

Leia o comando por cláusulas; execute e interprete o resultado.

Alterações continuam sujeitas a PK, UNIQUE, CHECK e FK

UNIQUE

Trocar e-mail por valor já usado é rejeitado.

CHECK

Definir estoque negativo é rejeitado.

FOREIGN KEY

Apontar pedido para cliente inexistente é rejeitado.

DELETE remove linhas selecionadas por uma condição

```
DELETE FROM cliente  
WHERE id_cliente = 17;
```

```
-- RESTRICT pode bloquear se houver pedidos.  
-- O erro revela uma dependência ainda existente.
```

Leia o comando por cláusulas; execute e interprete o resultado.

Excluir o pai pode bloquear, propagar ou desfazer o vínculo

RESTRICT

Rejeita a exclusão enquanto houver filhos.

CASCADE

Exclui também os filhos dependentes.

SET NULL

Preserva o filho e torna a FK nula.

Use SELECT com o mesmo WHERE antes de UPDATE ou DELETE

```
SELECT id_pedido, status, criado_em  
FROM pedido  
WHERE status = 'CANCELADO'  
      AND criado_em < '2026-01-01';
```

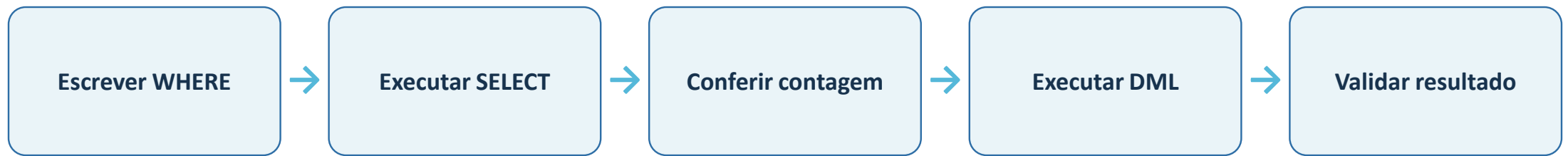
```
DELETE FROM pedido  
WHERE status = 'CANCELADO'  
      AND criado_em < '2026-01-01';
```

Leia o comando por cláusulas; execute e interprete o resultado.

Escolha pelo alvo, pelas dependências e pela necessidade de filtro

Critério	DELETE	TRUNCATE
WHERE	Aceita	Não aceita
Alcance	Algumas ou todas as linhas	Todas as linhas
Categoria	DML	DDL
FKs	Aplica ações linha a linha	Pode ser impedido por referências
Estrutura	Preservada	Preservada

Rotina segura para UPDATE e DELETE



04

Exemplo resolvido e exercícios

O roteiro integra criação, alteração, povoamento, mudanças e interpretação de erros, seguido de tarefas de dificuldade crescente.

Criar a base e respeitar a ordem das dependências

Etapa			Comando			Verificação		
1			CREATE DATABASE + USE			SELECT DATABASE()		
2			CREATE cliente e produto			SHOW TABLES		
3			CREATE pedido			SHOW CREATE TABLE		
4			CREATE item_pedido			SHOW INDEX		
5			Testar uma FK órfã			Erro esperado		

Alterar a estrutura sem contradizer dados existentes

```
ALTER TABLE produto  
ADD COLUMN ativo BOOLEAN NOT NULL DEFAULT TRUE;
```

```
ALTER TABLE produto  
ADD CONSTRAINT ck_produto_ativo  
CHECK (ativo IN (0,1));
```

```
SHOW CREATE TABLE produto;
```

Leia o comando por cláusulas; execute e interprete o resultado.

Povoar pais antes dos filhos e interpretar falhas

```
INSERT INTO cliente (nome,email) VALUES  
('Ana Lima','ana@exemplo.br'),  
('Bruno Reis','bruno@exemplo.br');
```

```
INSERT INTO pedido (id_cliente) VALUES (999);  
-- ERRO esperado: não existe CLIENTE 999.  
-- Correção: usar uma chave realmente persistida.
```

Leia o comando por cláusulas; execute e interprete o resultado.

Atualizar e excluir com alvo verificável

```
SELECT id_produto, nome, preco_atual  
FROM produto WHERE id_produto = 2;
```

```
UPDATE produto SET preco_atual = 19.90  
WHERE id_produto = 2;
```

```
DELETE FROM cliente WHERE id_cliente = 1;  
-- Se houver pedido, RESTRICT preserva o histórico.
```

Leia o comando por cláusulas; execute e interprete o resultado.

Interpretar o erro pelo contrato violado

Mensagem/sintoma	Contrato	Pergunta diagnóstica
Duplicate entry	PK ou UNIQUE	Qual valor já existe?
Cannot add/update child	FOREIGN KEY	O pai existe e os tipos coincidem?
Check constraint violated	CHECK	Qual predicado ficou falso?
Column cannot be null	NOT NULL	O valor foi omitido ou ficou nulo?
Data truncated/out of range	Tipo/domínio	Tipo e modo SQL são adequados?

Crie e altere um schema acadêmico

- Crie o schema universidade e selecione-o.
- Crie ALUNO, DISCIPLINA e MATRICULA com InnoDB.
- Inclua PKs, FKs, UNIQUE e CHECK necessários.
- Adicione em DISCIPLINA a coluna carga_horaria com ALTER TABLE.
- Inspeccione a definição final e registre a ordem dos comandos.

Povoe a base respeitando dependências

- Insira três alunos e três disciplinas com inserção múltipla.
- Recupere ou consulte as chaves geradas.
- Insira matrículas válidas.
- Provoque uma matrícula com aluno inexistente.
- Interprete o erro e corrija o comando sem desabilitar a FK.

Atualize e exclua com segurança

- Selecione previamente as matrículas ATIVAS de 2026.
- Atualize apenas esse conjunto para EM_CURSO.
- Tente excluir uma disciplina com matrículas.
- Explique o comportamento de RESTRICT ou CASCADE escolhido.
- Demonstre por que a mesma instrução sem WHERE seria perigosa.

Entregue scripts DDL e DML para uma clínica

- Crie PACIENTE, MEDICO, CONSULTA, PROCEDIMENTO e CONSULTA_PROCEDIMENTO.
- Evolua o schema com pelo menos dois ALTER TABLE.
- Povoie pais e filhos com INSERT simples e múltiplo.
- Inclua UPDATE e DELETE precedidos de SELECT equivalente.
- Provoque e documente quatro erros de integridade.
- Finalize com SHOW CREATE TABLE e SHOW INDEX das estruturas principais.

Recuperação ativa: responda sem consultar os slides

- Qual diferença prática existe entre DELETE, TRUNCATE e DROP?
- Por que a ordem de criação e inserção segue as FKs?
- Que garantias vêm de PK/UNIQUE e quais vêm de índices comuns?
- Por que AUTO_INCREMENT pode apresentar lacunas?
- Como reduzir o risco de UPDATE ou DELETE sem WHERE?
- Que perguntas transformam uma mensagem de erro em diagnóstico?

O essencial da T06 em cinco ideias

- DDL cria, altera e remove objetos do schema.
- Chaves geram garantias e índices associados, mas índice comum não é constraint.
- AUTO_INCREMENT fornece identidade, não sequência sem lacunas.
- DML deve respeitar tipos, constraints e dependências referenciais.
- O alcance de UPDATE e DELETE precisa ser verificado antes da execução.

Scripts confiáveis tornam cada mudança explícita e testável

